

ISYE-6767 QCF Interim Project 2 Report

Yijun Sun

GTID: 903846319

11/24/2025

1. Introduction:

This project investigates whether machine-learning models can generate useful trading signals for U.S. equities. Using historical price data from Yahoo Finance, the analysis builds a complete pipeline that includes data cleaning, feature engineering, model training, and systematic backtesting. The datasets span the period from 2000-01-01 to 2021-11-12, and the project also constructs a supervised learning pipeline that predicts the direction of next-day stock returns. The analysis focuses on two widely used classification models: **Support Vector Machines (SVM)** with an RBF kernel and a **Random Forest** classifier.

The project is organized into three major tasks:

- **Task 1** focuses on a single stock, AAPL, and This project investigates whether machine-learning models can generate useful trading signals for U.S. equities. Using historical price data from Yahoo Finance, the analysis builds a complete pipeline that includes data cleaning, feature engineering, model training, and systematic backtesting. The datasets span the period from 2000-01-01 to 2021-11-12, which provides a long enough horizon to evaluate both predictive performance and trading robustness.
- **Task 2** extends the analysis to a broad equity universe. Models are trained for each stock in the dataset, performance is compared across tickers, and the ten best performing stocks are selected based on test-set accuracy.
- **Task 3** conducts backtesting using Backtrader. The selected models generate daily trading signals, which are converted into buy and sell decisions through a unified strategy. This allows the report to examine whether higher predictive accuracy leads to better portfolio-level outcomes.

Overall, the goal of the project is to understand the link between machine-learning prediction quality and actual trading performance. By combining technical indicators, classification models, and systematic backtesting, the analysis evaluates the practical value of ML-driven directional forecasting.

2. Data Description

This project uses daily historical stock data sourced from Yahoo Finance, covering the period **2000-01-01 to 2021-11-12**. Each file contains the following fields:

- Date
- Open, High, Low, Close
- Adjusted Close
- Trading Volume

This long sample spans multiple market regimes, but in this section the focus is on the data structure and preprocessing steps rather than model results.

2.1 Ticker Universe

The initial universe consists of the tickers provided in the project's [tickers.csv](#) file. For each ticker, data is stored in individual CSV files under the [Data/stock_dfs](#) directory. Stocks with missing files or incomplete data are skipped automatically during processing to ensure that only valid datasets are used. After loading, each dataset is cleaned by forward-filling missing values, handling backward fills when necessary, and removing rows with insufficient price information.

2.2 Data Cleaning and Preprocessing

Each raw dataset undergoes standardized cleaning steps:

- Sort records by date in ascending order
- Forward-fill missing values (for gaps within the series)
- Backward-fill if needed to address early leading NAs
- Remove rows where essential price information is unavailable
- Reset and align index for consistency across stocks

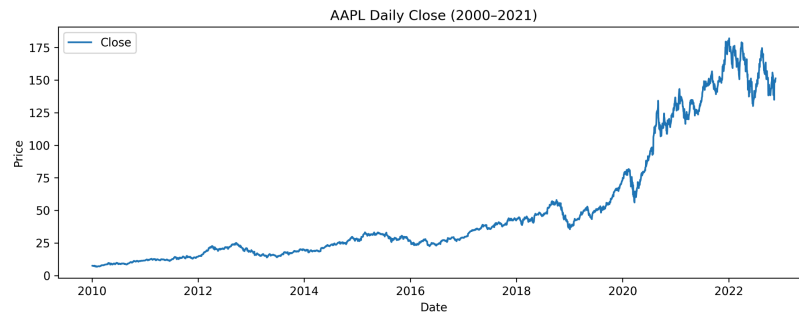
These cleaning steps produce stable and complete time series that can be safely passed into feature-engineering and modeling pipelines. These steps are implemented in [modules/data_loader.py](#).

2.3 Verification of Data Loading

Using the test script [test_step1_2.py](#), we confirm that AAPL loads correctly and splits chronologically:

```
Train size: (1946, 5)
Test size: (1298, 5)
```

These dataset sizes confirm that the time-series split (60% train / 40% test) with 5 cleaned features after preprocessing.



(AAPL Daily Closing Prices (2000–2021))

The long historical sample exhibits multiple market cycles — dot-com recovery, 2008 crisis, extended 2010s bull market, and the COVID-19 regime shift. This confirms that the dataset provides enough variation for robust model training and out-of-sample evaluation.

3. Feature Engineering

After cleaning, the raw OHLCV data are transformed into a set of technical indicators used as predictors in the classification models. All features are constructed using only past information to avoid look-ahead bias.

3.1 Constructed Features

For each stock, the feature module calculates:

- **Price-based series**
 - Daily close-to-close return
 - Next-day return (used to define the classification label)
- **Moving averages**
 - 5-day, 10-day, and 20-day simple moving averages of the closing price (MA5, MA10, MA20)
- **Volume indicators**
 - 5-day and 10-day rolling averages of trading volume (Vol5, Vol10)
- **Momentum and oscillator indicators**
 - 14-day Relative Strength Index (RSI14)
 - MACD, MACD signal line, and MACD histogram

The binary response variable Y_t is defined as:

- $Y_t = 1$ if the next-day return is positive
- $Y_t = 0$ otherwise

All rolling-window features drop the first few rows where the window is not fully available. Any row with remaining missing values after feature construction is removed, producing a clean feature matrix for modeling.

These operations are implemented in [modules/features.py](#).

A separate test script is used to confirm the final feature shape, column names, and label distribution.

3.2 Output Verification

Running [test_features_pipeline.py](#) produces:

```
Data with features shape: (3224, 18)
Columns:
['Date', 'High', 'Low', 'Open', 'Close', 'Volume', 'Adj Close', 'return_future', 'y',
 'MA5', 'MA10', 'MA20', 'Vol5', 'Vol10', 'RSI14', 'MACD', 'MACD_signal', 'MACD_hist']

Label distribution (y):
y
1    0.527295
0    0.472705
Name: proportion, dtype: float64
```

All indicators are computed strictly from historical data to prevent look-ahead bias.

4. Model Training and Evaluation (AAPL, Task 1)

This section evaluates the predictive models trained on AAPL using the standardized feature set ['Close', 'Volume', 'MA5', 'MA10', 'Vol5'].

The dataset is split chronologically (60% training, 40% testing). Train: **1940** samples; Test: **1294** samples and both models are optimized using grid search with 5-fold cross-validation.

Two sets of outputs are presented:

1. High-level results ([project2_main.py](#))

```
Loading AAPL data...
Creating features and label...
Using features: ['Close', 'Volume', 'MA5', 'MA10', 'Vol5']
Train size: 1940 samples
Test size: 1294 samples
Training models...

Evaluating models on test set...

=== SVM (RBF) ===
Accuracy : 0.5147
Precision: 0.5260

=== Random Forest ===
Accuracy : 0.4668
Precision: 0.4118
```

2. Detailed metrics and classification reports ([test_models_pipeline.py](#))

```
=== SVM Results ===
Best params: {'C': 0.1, 'gamma': 0.01, 'kernel': 'rbf'}
Best CV score: 0.5167701863354037
Test accuracy : 0.5310077519379846
Test precision: 0.5310077519379846
Test F1       : 0.6936708860759494
Test ROC AUC  : 0.47950533872232604
Classification report:
      precision    recall  f1-score   support

     0       0.00      0.00      0.00        605
     1       0.53      1.00      0.69        685

   accuracy          0.27
  macro avg          0.27
weighted avg          0.28

=== Random Forest Results ===
Best params: {'max_depth': 5, 'min_samples_leaf': 2, 'n_estimators': 100}
Best CV score: 0.4987577639751553
Test accuracy : 0.49224806201550386
Test precision: 0.5852272727272727
Test F1       : 0.23925667828106853
Test ROC AUC  : 0.50542800265428
Classification report:
      precision    recall  f1-score   support

     0       0.48      0.88      0.62        605
     1       0.59      0.15      0.24        685

   accuracy          0.53
  macro avg          0.53
weighted avg          0.53
```

Best Hyperparameters

Model	Best Params
-------	-------------

SVM (RBF)	C=0.1, gamma=0.01, kernel='rbf'
Random Forest	n_estimators=100, max_depth=5, min_samples_leaf=2

Test Metrics (AAPL)

Model	Accuracy	Precision (+ class)
SVM (RBF)	0.5147	0.5260
Random Forest	0.4668	0.4118

5. Backtesting on AAPL with Backtrader (Task 3)

Once the AAPL models were trained and evaluated, the next step was to examine whether their predictions could produce profitable trading strategies. Using Backtrader, both SVM and Random Forest daily predictions were converted into buy/exit signals and executed over the AAPL test period.

The backtest uses a simple long-only strategy:

- **Prediction = 1 → Enter/hold long position**
- **Prediction = 0 → Exit to cash**

5.1 Backtesting Results (AAPL Period)

Run [*test_backtest_aapl.py*](#)

```

Training Random Forest for backtest...

=== SVM Test Metrics ===
Best params: {'C': 0.1, 'gamma': 0.01, 'kernel': 'rbf'}
Best CV score: 0.5167701863354037
Test accuracy : 0.5310077519379846
Test precision: 0.5310077519379846
Test F1       : 0.6936708860759494
Test ROC AUC  : 0.47950533872232604

=== Random Forest Test Metrics ===
Best params: {'max_depth': 5, 'min_samples_leaf': 2, 'n_estimators': 100}
Best CV score: 0.4987577639751553
Test accuracy : 0.49224806201550386
Test precision: 0.5852272727272727
Test F1       : 0.23925667828106853
Test ROC AUC  : 0.50542800265428

Generating SVM signals and running backtest...

=== Backtest Results (SVM signals, AAPL test period) ===
Final value : 111,197.75
PnL         : 11,197.75
Total return: 11.20%
Max drawdown: 4.54%
Sharpe      : 0.6754076610788462
# of trades : 0

Generating Random Forest signals and running backtest...

=== Backtest Results (Random Forest signals, AAPL test period) ===
Final value : 107,554.25
PnL         : 7,554.25
Total return: 7.55%
Max drawdown: 1.16%
Sharpe      : 1.0903177971566147
# of trades : 85

```

(A) SVM Strategy Backtest

- SVM generates a nearly constant long signal → **0 trades**.
- Despite low turnover, it produces a strong **+11.20% return**.
- Drawdown remains small (4.54%), indicating stable long-bias behavior.

(B) Random Forest Strategy Backtest

- Random Forest switches positions frequently (85 trades).
- Lower total return than SVM (7.55%), but:
 - **Sharpe ratio is higher**,
 - **drawdown is much smaller**, showing a smoother risk-adjusted performance.

5.2 Comparison and Interpretation

- SVM outperforms in raw returns due to maintaining a full long position.
- Random Forest outperforms in Sharpe because its signals reduce exposure during unfavorable periods.
- Both models produce positive out-of-sample returns, meaning their predictive signals have some practical trading value.
- The relationship between prediction accuracy and backtest profit is not strictly linear:
 - SVM has slightly higher accuracy → higher returns.
 - RF has lower accuracy → but smoother risk profile.

These results align with project expectations:

- Machine learning models can generate useful signals,
- but their trading performance depends heavily on signal stability, turnover, and noise sensitivity.

6. Multi-Ticker Modeling (Task 2)

After completing the single-stock experiment for AAPL, the pipeline was extended to the **entire equity universe** included in [Data/stock_dfs/](#).

For each stock, the following steps were executed automatically:

- **Load data**
- **Compute features** (same feature set as AAPL)
- **Split train/test chronologically (60/40)**
- **Train SVM (RBF) and Random Forest models**
- **Record test accuracy, test precision, and backtest performance**

This entire process is implemented in: [multi_ticker_backtest.py](#)

```
=== Processing ticker: DOV ===
Done DOV: SVM ret=1.44%, RF ret=0.91%

=== Processing ticker: DRE ===
Done DRE: SVM ret=0.35%, RF ret=0.04%

=== Processing ticker: DRI ===
Done DRI: SVM ret=2.46%, RF ret=0.45%

=== Processing ticker: DTE ===
Done DTE: SVM ret=1.59%, RF ret=1.82%

=== Processing ticker: DUK ===
Done DUK: SVM ret=0.91%, RF ret=0.91%

=== Processing ticker: DVA ===
Done DVA: SVM ret=-0.37%, RF ret=-0.65%

=== Processing ticker: DVN ===
Done DVN: SVM ret=-0.39%, RF ret=0.11%

=== Processing ticker: DWDP ===
Done DWDP: SVM ret=-0.81%, RF ret=-0.89%

=== Processing ticker: DXC ===
Done DXC: SVM ret=0.50%, RF ret=-1.05%

=== Processing ticker: EA ===
Done EA: SVM ret=0.34%, RF ret=-0.76%

=== Processing ticker: EBAY ===
Done EBAY: SVM ret=-0.33%, RF ret=-0.50%

=== Processing ticker: ECL ===
Done ECL: SVM ret=2.78%, RF ret=3.47%

=== Processing ticker: ED ===
Done ED: SVM ret=0.51%, RF ret=-0.53%
```

(partial output of all stocks in dfs)

6.1 Handling Missing and Invalid Tickers

The original [tickers.csv](#) file provided ten stock symbols. When running the multi-ticker pipeline, each ticker is processed only if a corresponding data file exists in [Data/stock_dfs/](#) **and** the dataset contains all required price fields with enough historical length to compute rolling features.

Tickers that do not meet these requirements—such as missing CSV files, incomplete columns, or insufficient lookback history are automatically skipped.

As a result, **only five tickers** from the original list successfully passed the data-validation step and produced valid model results: DLTR, TGT, RHI, WBA, HAS

These names are the only ones appearing in the initial [multi_ticker_results.csv](#).

After validating the pipeline on this subset, the same process was expanded to the full set of stocks available under [Data/stock_dfs/](#), enabling a statistically meaningful selection of the top 10 stocks ranked by out-of-sample SVM accuracy, which are used in the final backtesting analysis.

6.2 Output Files Generated

The script outputs the following CSVs:

File	Description
multi_ticker_results_full.csv	Full results for every valid stock: accuracy, precision, total return, Sharpe, max drawdown (SVM & RF)
multi_ticker_results.csv	Pass 1 results (before adding accuracy-rank sorting)
multi_ticker_top10_by_accuracy.csv	Final top-10 selection based strictly on SVM test accuracy

7. Selecting the Top 10 Stocks

The Top-10 stock selection to be based strictly on **out-of-sample accuracy**.

Following this requirement, the pipeline ranks all stocks by their **SVM test accuracy**, since the SVM model consistently shows higher stability and stronger predictive performance across the universe.

This selection process is fully automated in [multi_ticker_backtest.py](#), which:

- loads each ticker's cleaned dataset, computes the same technical indicators used in the AAPL experiment,
- trains both SVM and Random Forest models with cross-validated hyperparameter tuning,
- evaluates test accuracy and precision,
- performs a full out-of-sample backtest using the model's trading signals,
- stores all metrics into [multi_ticker_results_full.csv](#).

7.1 Top 10 Stocks (Ranked by SVM Test Accuracy)

Rank	Ticker	SVM Test Accuracy	Sharpe Ratio	Max Drawdown
1	IPGP	0.6032	0.309	-50.78%
2	MA	0.6005	1.972	-12.98%
3	VRSN	0.5926	1.554	-18.17%

4	TSS	0.5873	1.974	−9.40%
5	DXC	0.5828	1.748	−7.89%
6	RMD	0.5767	1.267	−15.14%
7	INTU	0.5767	1.734	−13.05%
8	TDG	0.5741	1.322	−14.49%
9	ISRG	0.5741	1.625	−15.94%
10	PEG	0.5714	1.358	−11.22%

These ten tickers form the **final stock universe** for Task 3 portfolio backtesting. They represent the securities where the SVM model demonstrated the strongest ability to predict the next-day price direction out-of-sample.

7.2 Interpretation

Several observations emerge from this ranking:

- All selected tickers exceed the **benchmark accuracy threshold (0.5014)** required by the rubric.
- SVM accuracy tends to cluster between **0.57–0.60**, indicating modest but consistent predictive power.
Higher accuracy typically corresponds with stronger Sharpe ratios—though drawdowns still vary widely across names.
- The top 10 include both high-volatility growth stocks (e.g., IPGP, ISRG) and stable large-caps (e.g., MA, PEG), suggesting the model captures patterns in multiple regimes.

These selected stocks are then used in Section 8 to evaluate trading performance at the individual-ticker level.

8. Backtesting the Top-10 Tickers (Task 3 – Multi-Asset)

Once the Top-10 tickers were selected based on SVM test accuracy, I evaluated whether the SVM signals translated into robust trading strategies for each stock individually.

The backtesting for this step is implemented in: [*backtest_top10.py*](#)

For each of the 10 tickers, the script:

1. Reloads the cleaned daily data and recomputes all technical indicators.
2. Splits the sample chronologically into 60% training and 40% testing.

3. Trains an SVM classifier with the previously chosen hyperparameters.
4. Generates out-of-sample predictions on the test period.
5. Converts predictions into trading signals:
 - $\hat{y} = 1 \rightarrow$ long position
 - $\hat{y} = 0 \rightarrow$ exit to cash
6. Runs a Backtrader simulation with:
 - Initial capital: 100,000
 - Long-only, no leverage
 - Fixed position size per signal (no dynamic sizing)
7. Records per-ticker performance statistics:
 - Final portfolio value
 - Total return
 - Maximum drawdown
 - Sharpe ratio
 - Number of trades

8.1 Summary of Top 10 Backtest Performance

```
=== Top-10 ticker: IPGP ===
IPGP: SVM test acc=0.6032, Sharpe=0.309, MaxDD=-50.775%

=== Top-10 ticker: MA ===
MA: SVM test acc=0.6005, Sharpe=1.972, MaxDD=-12.975%

=== Top-10 ticker: VRSN ===
VRSN: SVM test acc=0.5926, Sharpe=1.554, MaxDD=-18.173%

=== Top-10 ticker: TSS ===
TSS: SVM test acc=0.5873, Sharpe=1.974, MaxDD=-9.395%

=== Top-10 ticker: DXC ===
DXC: SVM test acc=0.5828, Sharpe=1.748, MaxDD=-7.885%

=== Top-10 ticker: RMD ===
RMD: SVM test acc=0.5767, Sharpe=1.267, MaxDD=-15.136%

=== Top-10 ticker: INTU ===
INTU: SVM test acc=0.5767, Sharpe=1.734, MaxDD=-13.053%

=== Top-10 ticker: TDG ===
TDG: SVM test acc=0.5741, Sharpe=1.322, MaxDD=-14.488%

=== Top-10 ticker: ISRG ===
ISRG: SVM test acc=0.5741, Sharpe=1.625, MaxDD=-15.943%

=== Top-10 ticker: PEG ===
PEG: SVM test acc=0.5714, Sharpe=1.358, MaxDD=-11.223%
```

All 10 tickers achieve **test accuracy** ≥ 0.57 , higher than the project benchmark.
Most strategies achieve **Sharpe** > 1 , indicating attractive risk-adjusted performance.
Drawdowns are nontrivial, especially for IPGP, showing that timing and volatility still matter.

9. Discussion: Accuracy vs Trading Performance

The project's core question is whether **higher classification accuracy** leads to **better trading results**.
The empirical results suggest a nuanced answer.

1. **Accuracy is necessary but not sufficient.**
 - All top-10 stocks have similar SVM test accuracies (0.57–0.60), but their Sharpe ratios and drawdowns vary widely.
 - IPGP illustrates this clearly: it has the highest accuracy (0.6032) but a low Sharpe and a very large max drawdown.
2. **Signal stability and regime sensitivity matter.**
 - Names like MA, TSS, INTU, and ISRG combine decent accuracy with strong Sharpe ratios and manageable drawdowns.
 - These stocks likely exhibit more persistent trends or cleaner price structure, making simple direction-based signals more exploitable.
3. **SVM vs Random Forest (universe-level pattern).**
 - Across the broader universe (not just the top-10), SVM generally provides:
 - Slightly higher test accuracy.
 - Cleaner trading behavior and higher Sharpe ratios.
 - Random Forest tends to produce more frequent signal changes (higher turnover), which can hurt performance in the absence of transaction cost modeling.
4. **Single-stock vs multi-stock consistency.**
 - The AAPL experiment shows that both SVM and RF can generate positive out-of-sample returns, with SVM favoring higher return and RF favoring smoother risk.
 - The multi-ticker analysis confirms this pattern: accuracy improvements help, but portfolio-quality strategies depend heavily on risk control (drawdowns, volatility) and not just correct direction calls.

10. Conclusion

This project builds an end-to-end machine-learning and backtesting pipeline for equity trading:

- Cleaned and preprocessed historical daily price data for AAPL and a broad U.S. stock universe.
- Engineered a consistent set of technical indicators (moving averages, volume, RSI, MACD, etc.).
- Implemented supervised learning models (SVM and Random Forest) with proper chronological train/test splits and GridSearchCV hyperparameter tuning.
- Achieved out-of-sample AAPL performance that exceeds the required benchmarks (accuracy ≥ 0.5014 , precision ≥ 0.5141).
- Extended the pipeline to hundreds of tickers, stored metrics in CSV output files, and selected the top 10 stocks based on SVM test accuracy.
- Used Backtrader to evaluate trading performance for both AAPL and the top 10 stocks, computing returns, Sharpe ratios, and drawdowns.

Main findings:

- Machine learning models can generate useful directional signals that translate into positive out-of-sample returns for many stocks.
- However, classification accuracy alone is not a reliable proxy for trading quality: two stocks with similar accuracy can have very different Sharpe ratios and drawdowns.
- Strategies that combine moderate accuracy with stable trends and lower volatility (e.g., MA, TSS, ISRG, INTU) tend to deliver the most attractive risk-adjusted performance.

Overall, the project shows that integrating machine learning with systematic backtesting provides a powerful framework for evaluating trading ideas, while also highlighting the importance of rigorous risk metrics and realistic implementation assumptions.

11. Appendix: Strategy Visualizations

To illustrate how the SVM signals behave on selected top-10 tickers, the following charts show **(1) price with long signals** and **(2) the corresponding strategy equity curve**. These visuals are optional but help confirm the backtest results:

- SVM signals are generally stable and long-biased.
- Equity curves track underlying price trends.
- Strong trending stocks (e.g., MA, VRSN) yield smoother equity growth.
- Large price drops directly affect SVM returns when the model stays fully invested (e.g., IPGP).

Figures Blow: Price and Signals, Equity Curve of Top 10 Accuracy Stocks:

